

MAKE SURE TO HIT RECORD!!!

Speech

Intro - Kathy (2 mins) ... this is basically 3 mins

Hi all we are team Injured so we have me, Lawrence, Gobind, Samuel, Aqil, and Lachlan, and we're excited to show you what we've been working on. So our project is a **One-Click Job Application with [JobGen.AI](#)**. I'm sure many of you are in the same boat where we've applied for countless jobs across various platforms and it's incredibly time-consuming and repetitive. This problem has impacted a wide range of users, from students, grads, and many other job seekers, that are trying to further advance their career. So we've teamed up with JobGen.AI to create a more streamlined application process to allow users to apply for multiple jobs with just a click of a button.

So here's our systems diagram (**SHOW DIAGRAM**).

- So the user first clicks 'Apply' in the [JobGen.AI](#) chrome extension
- This extension would showcase all the details of the job listing and assess the candidate's suitability for the listing based on their resume
- it would try to detect application forms and then pass onto a DOM parser to detect form fields and identify field types, like name, email, employment history, or education.
- It would also manage file uploads for resumes and cover letters which would include addressing any limitations like type restrictions or quantity limits.
- Another thing is the autofill capability. It's designed to extract and store personal information from the user's resume so we can automatically populate various application form fields. It would autofill basic details like name and email, and later on, it would address more complex parts such as psychometric questions and scenario-based questions.
- Once the form is filled, we get sent to a preview screen of all the answers the user inputted. This allows the user to edit any autofilled information before submission.
- When that's submitted, we planned to log the application for tracking to allow the users to keep a record of their applications.

(**SHOW JIRA**) For this 1st sprint, we've been meeting up mostly every day to discuss our design decisions. Our focus has been to analyse the application form structures on LinkedIn and Seek. We've also been focusing on getting the APIs to detect form fields so we can autofill them, and also uploading files to a cloud storage service which has been proven

to be quite difficult at the moment. From that, I'll pass it over to Lawrence to showcase our user interface.

Lawrence: Thanks, Kathy and Hi all. One of the most crucial steps to get our extension to work was identifying and analysing the application forms for the websites we wanted our extension to work on.

First, I conducted a visual analysis of the forms using Canva. I took screenshots of the websites, which helped us easily identify the visual differences in the forms and the variations in the overall application process. For example, LinkedIn asks for your contact information first, then your resume, followed by employer questions before submission. In contrast, Indeed requests your resume first, assesses job relevance, and poses employer questions.

Our biggest challenge during this process was transitioning from a visual analysis to an HTML analysis. This was difficult because not all websites used standardised naming conventions for their HTML identifiers, complicating the automation process on some sites more than others. For instance, LinkedIn was relatively easy to analyse (I'll demonstrate this with some code). Still, sites like Seek, Indeed, Naukri, and Glassdoor often use dynamic HTML identifiers that change with every page refresh, making it hard to store them in an internal configuration for the extension to read.

(DEMO CODE)

I will now hand it off to Gobind, who will demonstrate our autofill feature.

1. 🧩 2-3 minutes discussing project design –
 - 📖 Explicitly introduce background information about the problem, and the users impacted by this problem.
 - 🔧 Referring to the above, discuss and justify the design decisions that you made during the sprint to best solve that problem.
 - 💡 You could, for example, explore the below and discuss how they address your problem and requirements:

- Show a **system architecture diagram** and some notable design decisions
- Discuss any complex algorithms, interesting APIs, or data storage mechanisms, that you used to achieve a feature, or
- Discuss and justify any design choices that you made against possible alternatives.

Lawrence -> form analyse (canva), form analyse on linkedin (code), the problem the code tackles: helps the extension to identify where to autofill, so we can automate the process and help users effectively and swiftly apply for jobs using the job gen ai extension.

- Will talk about the forms in all platforms

Gobind - linkedin autofill demo

- Will talk about issues with the resume upload

Samuel - autofill from api call. (2mins) (try to get api 3 working for demo too.)

- Will showcase api calls on mock form

Hi all i will be demonstrating how the API works on mock forms.

We currently have 2 Apis working. APi 1, which answers short text and checklists questions using information retrieved from jobgen.ai account and autofills it, and API 3 which handles specific long text-based questions. As you can see for api 3 when the api returns an ai generated answer it autofills it and stores it into it's datamap. This allows it to retrieve the answer to the same question faster in the future without having to call the api again. Here you can see it working in real time. And here you can see the answer being stored.

Aqil -> show j-unit tests

Lachlan -> talk about future sprint (1 minute)

Looking back on this first sprint we could have clarified the requirements with the client earlier on to avoid doubling up on work. Another improvement to keep in mind for future sprints is to spend more time making sure individual team members understand the task they have been assigned to avoid confusion and decreased productivity.

Now as we look ahead to sprint 2, we should be completing a major portion of user stories in the backlog to ensure we have enough time to fix bugs and create proper documentation for the code base for project handover.

The most crucial task going into sprint 2 will be incorporating the CV upload function using AWS.

- Go back to jira board -> backlog
- Talk about improvements, what to do next
- “As Gobind had mentioned, as of right now, we plan to prioritise the resume upload on all platforms”.

2. 8–10 minutes demonstrating features –

- For development projects:

-  This involves demonstrating the functionality that you have implemented by showing the usage of that functionality from the user’s perspective (i.e. by showing how the user would use a feature via the user interface).
-  Some features may not have a fully functional interface. However, do not spend a significant amount of time discussing code. You have been working on your project for several weeks, but your tutor hasn’t, so it will not be possible for them to understand and evaluate you on your code itself in the limited time you have for your demo. If you do not have a functional interface for any features, you may demonstrate that feature on the command line or by showing the backend output but ensure that it will be easy to understand in a non-technical manner.
-  You should spend more time on features that deliver value to solving the problem and requirements. Typically, features like user logins, account management, and basic data ingestion might not provide strong benefits. Focus on other features that are unique and drive value to your product.

3.